The Code Deep Dive

Every package, every core function, the actual algorithms - read alongside the source, not instead of it. File paths and line-area references point at proofer/compare.py unless another file is named.

This is the companion to *How Fenxi Proofer Actually Works* - that one explained the shape of the thing in plain English. This one opens the hood: the exact packages, the exact data structures, and the exact algorithms behind the ten checks, with real code pulled straight from the repository.

1. The dependency stack - what's installed and why

Everything below is what's actually installed in this environment right now (checked with `pip list`), not just what `requirements.txt` asks for.

Package	Version	What it's doing here
`pymupdf`	1.28.2	The one library nearly everything else sits on top of. Opens PDFs, reads text with exact coordinates, reads vector drawing/paint commands, renders pages to pixels, and (new) converts a raw JPG/PNG into a real one-page PDF. Python binding for MuPDF, a C rendering engine.
`numpy`	2.5.3	Every image is a NumPy array (height x width x 3 uint8). All the pixel math - colour-space conversion, masking, distance calculations - is vectorised NumPy, not Python loops.
`opencv-python-headless`	5.0.0.93	Image processing: resizing, blurring, colour-space conversion, connected-component labelling (clustering changed pixels into regions), ORB feature detection + RANSAC (registration), perspective warping.
`scikit-image`	0.26.0	Used alongside OpenCV for a couple of specific pixel operations (structural comparisons) where its implementation was the better fit.
`scipy`	1.18.1	One precise job: `scipy.optimize.linear_sum_assignment` - the Hungarian algorithm, used when two files have the same page count but the pages don't line up 1-to-1 (re-ordered pages).
`zxing-cpp`	3.1.1	Barcode decoding. Python binding for ZXing-C++, reads EAN/UPC/Code128/DataMatrix/QR/PDF417/etc, returns payload, format, corner points and a validity flag in one call.
`fastapi`	0.141.1	The web framework behind app.py: request routing, request validation via Pydantic models, file uploads, JSON responses.
`uvicorn`	0.53.0	The ASGI server that actually runs FastAPI and listens on 127.0.0.1:8000.

Package	Version	What it's doing here
`python-multipart`	0.0.32	FastAPI's multipart/form-data parser - required for file uploads (`UploadFile`) to work at all; FastAPI raises at startup without it.
`pydantic`	2.13.5	Request/response validation - every POST body (review decisions, settings, sign-off) is a typed `BaseModel`, so a malformed request is rejected before it reaches any proofing logic.
`rapidocr-onnxruntime`	1.4.4	Optional OCR, wrapping ONNX Runtime with PaddleOCR-family detection + recognition + orientation-classifier models. Only used to caption a region the pixel check already flagged, or to read text on image-only artwork. Bundled recognition model is Chinese+English only - see section 9.
`onnxruntime`	1.30.0	The actual inference engine RapidOCR runs its three ONNX models on (detection, orientation classification, recognition). CPU-only here.
`pyspellchecker`	0.9.0	Powers proofer/spelling.py. Note: this is installed and working, but it is NOT listed in requirements.txt - a real gap worth fixing so a clean install doesn't silently lose spell-check.
`Pillow, pyclipper, shapely`	12.3.0 / 1.4.0 / 2.1.2	Transitive dependencies of rapidocr-onnxruntime (image handling and polygon geometry for its text-detection post-processing) - not imported directly by proofer's own code.

A deliberate non-dependency

python-docx is **not** a dependency, even though the tool reads .docx copy decks. proofer/copydeck.py opens the .docx as a plain ZIP file (which is all a .docx actually is) and parses `word/document.xml` with the standard library's own xml.etree - one less dependency, and one less thing that can drift out of sync with a new Word version.

2. The data model: three dataclasses everything flows through

Almost the entire engine's output funnels through two dataclasses (proofer/compare.py) plus the tolerance object every check reads from.

Config - every tolerance, in one place

proofer/compare.py

```
@dataclass
class Config:
    dpi: int = 300
    delta_e: float = 3.0        # a pixel is "changed" above this CIE76 dE
    min_area_px: int = 6        # ignore specks smaller than this
    merge_mm: float = 0.8       # changed pixels closer than this belong to one region
    move_tol_mm: float = 0.25
    move_search_mm: float = 15.0
    move_max_residual: float = 0.06
    barcode_shift_mm: float = 0.5
    barcode_min_contrast: float = 0.5
    ocr: bool = True
    register: bool = True       # detect + undo a whole-page shift/scale/rotation first
    ...  # 25 fields total
```

Nothing in the codebase hardcodes a threshold inline - every number a check compares against is a field here, which is exactly what makes profiles (section 12) possible: a profile is just `dataclasses.replace(Config(), **overrides)`.

Finding - one difference, fully described

```
@dataclass
class Finding:
    category: str    # text | barcode | graphic | layout | color | format | structure | info |
braille | layer
    kind: str        # e.g. "changed", "digits_mismatch", "language_mismatch"
    severity: str    # critical | major | minor | info
    page: int
    message: str     # the sentence a human reads
    a_text: str = ""
    b_text: str = ""
    bbox_a: list | None = None   # PDF points [x0, y0, x1, y1], reference
    bbox_b: list | None = None   # same, sample
    detail: dict = field(default_factory=dict)   # everything check-specific lives here
    id: int = 0
    confidence: str = "high"    # high = read from the file; medium = inferred
```

Every one of the ~70 functions in compare.py that finds something returns `list[Finding]`. That uniform shape is what makes merging possible: the pipeline never needs to know which check produced a finding to file it, sort it, render it, or annotate it onto a PDF - it only ever needs these ten fields.

Result - one whole proof

```
@dataclass
class Result:
    findings: list
    pages: list             # per-page render data for the HTML report
    warnings: list
    coverage: list = field(default_factory=list)   # per page: which checks ran / were skipped,
and why
    attention: list = field(default_factory=list)  # not a difference, but a human should look

    @property
    def verdict(self) -> str:
        # PASS / REVIEW / FAIL, computed from the worst surviving severity
```

3. From an HTTP POST to a running proof

app.py's job-creation endpoint is intentionally thin - it validates, converts, and hands off immediately, so the HTTP request returns in milliseconds even though the proof itself takes seconds to minutes:

app.py

```
@app.post("/api/jobs")
async def create_job(reference: UploadFile = File(...), sample: UploadFile = File(...),
                     dpi: int = Form(300), ocr_on: bool = Form(True), ...):
    ref, smp = await _pdf(reference, "Reference"), await _pdf(sample, "Sample")
    ...
    return dict(id=jobs.create(ref, smp, reference.filename, sample.filename, dpi, ocr_on, ...))
```

`_pdf()` is the upload gate - size limit (200 MB), then a content sniff, not a filename check:

app.py

```
async def _pdf(up: UploadFile, label: str) -> bytes:
    data = await up.read()
    if len(data) > MAX_UPLOAD:
        raise HTTPException(413, ...)
    if is_pdf(data):
        return data
    if is_image(data):
        return to_pdf_bytes(data)   # wrap JPG/PNG into a real one-page PDF - see section 10
    raise HTTPException(400, f"{label} must be a PDF, JPG or PNG file")
```

`jobs.create()` (proofer/jobs.py) writes the two files to `results/jobs/<id>/`, writes a `status.json` with state `queued`, and submits the real work to a thread pool:

proofer/jobs.py

```
_pool = ThreadPoolExecutor(max_workers=int(os.environ.get("PROOFER_WORKERS", "2")))
...
def create(ref, sample, ref_name, sample_name, dpi=300, ...):
    job_id = time.strftime("%y%m%d%H%M%S") + uuid.uuid4().hex[:4]
    ...
    _write(d / "status.json", dict(state="queued", ...))
    _pool.submit(_run, job_id)
    return job_id
```

The job id itself doubles as a sortable timestamp (`YYMMDDHHMMSS` plus 4 random hex characters to break ties), which is why `listing()` can sort jobs by `created` without a database. The browser polls `GET /api/jobs/{id}` for `status.json`'s `progress` field until state flips to `done` or `error`.

Why a thread pool and not a process pool, here

Two workers by default (`PROOFER_WORKERS`), threads not processes. That works because the heavy lifting - OpenCV, NumPy, PyMuPDF's rendering - is C/C++ code that releases Python's GIL while it runs, so two proofs really do run in parallel on separate CPU cores despite being Python threads. Batch mode (proofer/batch.py) uses a `ProcessPoolExecutor` instead, because it's a short-lived CLI process proofing many pairs, not a long-lived server juggling shared in-memory job state.

The audit log is a real hash chain, not just a list

Every review decision and sign-off is appended to `audit.jsonl` with this shape:

proofer/jobs.py

```
def _append(job_id, entry):
    prev = (_entries(job_id) or [{"hash": "0"*64}])[-1]["hash"]
    entry = {**entry, "ts": ..., "prev": prev, "engine": __version__}
    entry["hash"] = hashlib.sha256((prev + json.dumps(entry,
sort_keys=True)).encode()).hexdigest()
    # ... append entry as one line of JSON to audit.jsonl
```

Each entry's hash is computed over its own content *plus* the previous entry's hash - the same construction a blockchain uses. `audit()` walks the file and recomputes every hash from scratch; if one entry was hand-edited after the fact, its recomputed hash won't match what's stored, every hash after it goes wrong too, and `intact` flips to `False` for the whole chain. There's no cryptographic signing here (no private key), so this proves tampering happened - it doesn't prove who did it. That distinction matters if this is ever positioned for a regulated audit.

4. compare() - the master function

Everything above exists to eventually call one function: `compare(path_a, path_b, cfg)` in proofer/compare.py, ~200 lines that orchestrate every check. Its shape:

```
def compare(path_a, path_b, cfg=None, ...):
    A, B = open_pdf(path_a), open_pdf(path_b)      # imaging.py: PDF or JPG/PNG, either way a real
PDF
    if len(A) != len(B): findings.append(... "page_count" ...)
    pairs, lost, gained = _align_pages(A, B, cfg)   # which page matches which - section 5
    for p, pj in pairs:
        pa, pb = A[p], B[pj]
        T = align.estimate(pa, pb, img_a, img_b, ...) if cfg.register else None   # section 6
        if T is not None:
            img_b = align.render_aligned(pb, T["H"], ...)   # re-render sample into ref's pixel
frame
        text_f  = compare_text(pa, pb, p, cfg, k_size)        # section 7
        lang_f  = compare_language(pa, pb, p)                 # section 8 - added this session
        code_f  = compare_barcodes(img_a, img_b, pa, pb, p, cfg, ...)   # section 9
        color_f = compare_colorants(pa, pb, p, H)
        brl_f   = compare_braille(pa, pb, p)
        layer_f = compare_layers(A, B, pa, pb, p)
        struct_f = compare_structure(pa, pb, p, cfg, raster, H) + compare_inks(pa, pb, p)
        sep_f   = compare_separations(pa, pb, p, cfg, H)
        vis_f, mask, vstats = compare_visual(img_a, img_b, pa, pb, p, explained, ...)   # section
10
        findings += text_f + lang_f + code_f + color_f + brl_f + layer_f + struct_f + vis_f
    findings = _merge_confirmations(findings)       # one real difference -> one finding
    findings = profiles.apply(findings, settings)   # variable-data / lenient rules
    return Result(findings, pages, warnings, coverage, attention)
```

(Simplified - the real function also builds the `explained` list that stops one difference being reported twice, described in the previous document's section 5 and expanded in section 10 below.)

5. Pairing pages: a dynamic-programming sequence alignment

`_align_pages()` has to answer "which reference page is which sample page" before anything else can run - get this wrong and every later check compares the wrong pages. Three cases:

- **Same count, ordinary case:** page i matches page i. Cheap: only compute a full similarity check if some page doesn't resemble its same-position partner.
- **Same count, but reordered:** build the full N×N similarity matrix and solve it with `scipy.optimize.linear_sum_assignment` - the Hungarian algorithm, which finds the assignment of pages to pages that maximises total similarity in polynomial time (not by trying every permutation).
- **Different counts** (a page inserted, removed, or a whole extra leaflet panel): this is exactly the textbook sequence-alignment problem - the same shape as DNA alignment or a text diff algorithm. Solved with the identical dynamic-programming recurrence:

proofer/compare.py - _align_pages

```
best[i][j] = max(
    best[i-1][j],                                       # skip a reference page (it was
removed)
    best[i][j-1],                                       # skip a sample page (it was
inserted)
    best[i-1][j-1] + sim(i-1, j-1) if sim >= threshold else 0  # match these two pages
)
```

`sim(i, j)` - `_page_sim()` - is itself two different measures depending on what's on the page: if both pages have real text, it's a `difflib.SequenceMatcher` ratio on the first 2500 characters; if neither does (image-only), it's 1 minus the normalised pixel difference of a tiny 24×32 greyscale thumbnail (cheap enough to compute for every page pair without ever rendering at full resolution).

Why this matters

This is why an inserted page is reported **once** ("reference page 4 has no match") instead of silently comparing reference page 4 against sample page 4, reference page 5 against sample page 5, and so on - which would produce a wall of false differences for every page after the insertion point.

6. Registration: RANSAC under the hood

Covered conceptually in the companion document; here's the actual estimator, align.py's `estimate()`:

proofer/align.py

```
def estimate(pa, pb, img_a, img_b, dpi, live, move_tol_mm=0.25):
    src, dst = _word_points(pa, pb) if live else (empty, empty)     # words on both pages exactly
once
    if len(src) < 8:
        src, dst = _feature_points(img_a, img_b, dpi)               # ORB keypoints instead
    if len(src) < 8: return None                                    # not enough evidence either
way
    still = np.linalg.norm(src - dst, axis=1) < tol
    if still.mean() >= 0.5: return None                             # already registered - do
nothing
    M, inliers = cv2.estimateAffinePartial2D(src, dst, method=cv2.RANSAC, ...)
    if inliers.mean() < 0.6:                                        # weak affine fit: try full
perspective
        H, inliers = cv2.findHomography(src, dst, cv2.RANSAC, 4.0)
    if inliers.sum() < 8 or inliers.mean() < 0.5: return None        # not enough of the page
agrees
    return dict(H=H, scale=..., rotation=..., dx_mm=..., dy_mm=..., inliers=..., ratio=...)
```

What RANSAC is doing here, concretely

RANSAC (Random Sample Consensus) repeatedly picks a small random subset of the matched point pairs, fits a transform to just that subset, then counts how many of the *other* points agree with it within a tolerance (the "inliers"). It keeps the transform with the most inliers. The effect: if 95% of the page shifted by the same 2mm and one logo genuinely moved on its own, RANSAC finds the 2mm shift (backed by nearly every point) and correctly reports the logo's own movement as an outlier - i.e. a real, separate finding - instead of the outlier corrupting the whole-page estimate.

Two independent evidence sources, by design

`_word_points()` matches words that appear *exactly once* on each page (a `Counter` filters out repeated words like "mg" or "the"), using their text as the matching key and their centre point as the coordinate. `_feature_points()` is the fallback for image-only pages or photos: ORB (Oriented FAST and Rotated BRIEF) keypoints detected on both renders, matched with a brute-force Hamming-distance matcher and Lowe's ratio test (a match is only kept if it's convincingly better than the second-best candidate, at the 0.75 threshold), then handed to the same RANSAC step.

7. The text engine: geometry clustering + diff

Step 1 - turn words into paragraphs (no help from the PDF's own structure)

PDFs don't reliably say "these words are one paragraph" - that depends on the order the authoring tool wrote drawing commands in, which differs between identical-looking exports. `_text_blocks()` rebuilds paragraphs from pure geometry, in two clustering passes using a union-find (disjoint-set) structure:

proofer/compare.py - _text_blocks

```
def same_line(a, b):     # two word boxes belong on one line
    overlap = min(a.y1,b.y1) - max(a.y0,b.y0)      # vertical overlap
    gap = max(a.x0,b.x0) - min(a.x1,b.x1)           # horizontal gap
    return overlap >= 0.6 * min(height_a, height_b) and gap <= 0.9 * max(height_a, height_b)

def same_para(a, b):     # two lines belong in one paragraph
    gap = max(a.y0,b.y0) - min(a.y1,b.y1)           # vertical gap between lines
    xoverlap = ...                                   # lines must actually overlap horizontally
    return gap <= 0.35 * min(height_a, height_b) and xoverlap >= 0.3 * min(width_a, width_b) \
           and 0.75 <= height_a / height_b <= 1.33   # similar text size
```

Both passes reuse the same `_cluster()` helper: sort boxes by vertical position, sweep down the page unioning any two whose gap satisfies the rule above, and stop comparing once the vertical gap exceeds a generous cutoff - `O(n log n)` in practice instead of comparing every word to every other word.

Step 2 - match paragraphs between the two files

`_match_blocks()` scores every reference-paragraph/sample-paragraph pair with `difflib.SequenceMatcher` (Python's own Ratcliff-Obershelp string similarity), with a small bonus for being near the same position on the page, then greedily accepts the best-scoring pairs first, so a paragraph can't be claimed twice:

```
score = sm.ratio() + 0.2 * max(0, 1 - distance_between_boxes / (30 * MM))
# sort all candidate pairs by score, descending; take each one if both sides are still free
```

Anything left over after that gets one more chance: if a leftover reference paragraph and a leftover sample paragraph occupy almost the same rectangle (IoU > 0.3), they're paired anyway - "rewritten wholesale" rather than "paragraph deleted here, unrelated paragraph added there".

Step 3 - diff word by word, decide severity

Matched paragraphs go through a word-level diff; `_text_severity()` is where "a changed digit is always critical" actually lives:

```
def _text_severity(a, b):
    if re.search(r'\d', ' '.join(a+b)): return 'critical'         # any digit involved -> critical
    if strip_punct(a) == strip_punct(b):
        return 'major' if has_key_symbol(a+b) else 'minor'         # e.g. dropped a ° or % ->
major
    return 'major'
```

8. compare_language() - built this session

Sits right next to compare_text in the file, and deliberately doesn't use OCR or a language-ID model - just the PDF's own Unicode text, bucketed by script:

proofer/compare.py

```
_SCRIPTS = [
    ("Latin", 0x0041, 0x024F), ("Cyrillic", 0x0400, 0x04FF), ("Greek", 0x0370, 0x03FF),
    ("Arabic", 0x0600, 0x06FF), ("Hebrew", 0x0590, 0x05FF), ("Devanagari", 0x0900, 0x097F),
    ("Thai", 0x0E00, 0x0E7F), ("Han", 0x4E00, 0x9FFF), ("Hiragana/Katakana", 0x3040, 0x30FF),
    ("Hangul", 0xAC00, 0xD7A3),
]

def _script_counts(text):
    counts = Counter()
    for ch in text:
        for name, lo, hi in _SCRIPTS:
            if lo <= ord(ch) <= hi:
                counts[name] += 1; break
    return counts

def compare_language(pa, pb, page_no):
    ca, cb = _script_counts(pa.get_text()), _script_counts(pb.get_text())
    na, nb = sum(ca.values()), sum(cb.values())
    if na < 20 or nb < 20: return []                    # too little alphabetic text to be
confident
    dom_a, cnt_a = ca.most_common(1)[0]
    dom_b, cnt_b = cb.most_common(1)[0]
    if dom_a == dom_b or cnt_a/na < 0.6 or cnt_b/nb < 0.6: return []   # same script, or genuinely
mixed
    return [Finding("text", "language_mismatch", "critical", page_no,
        f"the reference's text is {dom_a} script but the sample's text is {dom_b} script...",
        a_text=dom_a, b_text=dom_b, confidence="high")]
```

`ord(ch)` gives a character's Unicode code point; checking it against known script ranges is exact and instant - no model, no training data, no network call. The 20-character and 60%-dominance minimums exist so a page that's legitimately multilingual (a few French words in an otherwise-English panel) doesn't trip the check.

Wired into the main loop right where `compare_text` runs, gated the same way - live text only, not on image-only pages:

```
text_f = [] if no_text else compare_text(pa, pb, p, cfg, k_size)
lang_f = [] if no_text else compare_language(pa, pb, p)
if lang_f:
    text_f = []   # the whole page is a different language: a word-by-word diff would just be
noise
...
explained = [[0, 0, pa.rect.width, pa.rect.height]] if lang_f else []   # suppress per-glyph
visual noise too
```

Verified, not just written

Tested with real generated PDFs (English reference, Russian sample, same layout): fires exactly one critical finding - "the reference's text is Latin script but the sample's text is Cyrillic script" - instead of ~50 word-level diffs plus a wall of garbled OCR captions on every changed region. Re-ran the full 82-defect answer key afterward: still 82/82 detected, 0/17 false alarms on controls, no regression.

Known limit: this only sees the PDF's real text layer. On image-only/scanned artwork there's no text layer to read, so this check doesn't fire there at all - and OCR currently can't fill the gap for non-Latin scripts (next section).

9. Barcodes: decode twice, cross-check the printed digits

Two decode passes, deliberately asymmetric-safe

proofer/compare.py

```
def _read_codes(img, dpi):
    out = _decode(img, dpi)                                  # pass 1: at render resolution
    up = cv2.resize(img, None, fx=1.6, fy=1.6, interpolation=cv2.INTER_CUBIC)
    for c in _decode(up, dpi * 1.6):                         # pass 2: upscaled 1.6x
        if not c.valid: continue                             # pass 2 may only ADD good symbols,
        if c not already found: out.append(c)                # never invent a broken one
    return out
```

A small or tightly-printed symbol that fails at 300 dpi sometimes succeeds once upscaled (zxing-cpp's own detector has a minimum practical module size); running both passes on *both* files means a difference in what decodes is itself meaningful, not an artifact of only checking one resolution.

The printed digits vs. the encoded payload

A barcode almost always has human-readable digits printed beside it. `_printed_digits()` reads them from the PDF text layer inside a padded zone around the barcode's decoded bounding box; `_ocr_printed_digits()` is the OCR fallback for image-only artwork, explicitly marked medium confidence by the caller. Either way, compare_barcodes checks that those digits equal what the bars actually decode to - catching the specific, common defect where someone edits the printed number but the underlying barcode graphic doesn't change (or vice versa).

Print-quality proxies, named honestly as proxies

```
def _print_quality(img, bbox, dpi):
    lo, hi = np.percentile(gray, 2), np.percentile(gray, 98)   # robust to a few outlier pixels
    return dict(contrast=hi-lo, min_reflectance=lo, max_reflectance=hi)

def _quiet_zone_ok(img, bbox, dpi, margin_mm):
    # looks at the strip immediately left/right of the barcode for anything non-blank
    return (gray_in_that_strip < 200).mean() <= 0.1
```

Both docstrings say plainly that this is *not* ISO/IEC 15416 grading (that needs multiple scanlines and a modulation/defect metric) - it's a single bounded measurement built to catch the most common real failure (bars and background too close in tone, or a quiet zone blocked by other print content), cheaply and without a

certified scanner.

10. The visual check: colour science, then connected components

Why CIE-Lab and not raw RGB

```
def rgb2lab(img):
    a = img.astype(np.float32) / 255.0
    return cv2.cvtColor(a, cv2.COLOR_RGB2Lab)
```

RGB distance doesn't match human colour perception - equal RGB distances can look wildly different depending on the colour. Lab space is built so that Euclidean distance between two colours (ΔE) roughly tracks how different they *look* to a human eye, which is why every colour tolerance in the whole engine (`delta_e`, `raster_delta_e`, ...) is a ΔE number, not an RGB one. The docstring notes OpenCV's conversion matches scikit-image's to within 0.3 ΔE at roughly 4x the speed - the reason OpenCV does this job instead of scikit-image, even though both are installed.

Three tolerance regimes, chosen per page

- **aligned** (both files share a pixel grid exactly - e.g. two flattens at the same dpi): full-sensitivity ΔE, no blur. This is what catches 4pt text differences.
- **raster** (image-only, or resampled by registration): two independent renders of identical vector art still differ by a little resampling noise, so a blurred comparison is used almost everywhere - *except* in regions measured as locally flat (near-constant tone in both renders), which get the tight threshold back, because a flat region genuinely can't drift between two renders of the same file. That's specifically what catches a subtle solid-fill recolour that the loose, noise-safe threshold alone would miss.
- **photo** (a phone photo of the pack): both thresholds loosened further and the minimum region size raised 40x, because a photo's own lighting and lens softness dwarf print-level differences.

From a difference mask to named findings

```
n, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)
keep = stats[:, AREA] >= cfg.min_area_px      # drop specks
grown = cv2.dilate(mask, ellipse_kernel(2*r+1))   # merge pixels within cfg.merge_mm of each other
```

Each surviving connected component is one candidate finding. It's then classified: does sliding it by some (dx, dy) make it match the reference almost perfectly (`move_max_residual` = at most 6% still disagreeing after the slide)? Then it's `moved`, found by `_find_shift()` searching a local window via normalised cross-correlation. Otherwise it's classified by comparing average colour/edge signature on each side: `added`, `removed`, `recoloured`, `restyled`, or a plain `changed`.

Nothing is double-counted: the explained-region mechanism

Before compare_visual ever runs, every bounding box already claimed by the text, barcode, colour, braille or layer checks (plus, now, a whole-page language mismatch - section 8) is collected into one `explained` list, padded a few points, and subtracted from consideration. Only pixel changes nobody has already explained get reported as new visual findings - this is the single mechanism that turns "ten independent checks" into "one

finding per real difference" instead of a pile of duplicates.

11. Colour, ink plates, layers, Braille - quick technical notes

Colour (compare_colorants)

Reads fill/stroke colour operators directly out of the PDF's own drawing commands (`page.get_drawings()`) - a genuine colour-value read, not a pixel guess, hence `confidence="high"` even before the visual check runs at all.

Ink separations (compare_separations, compare_inks)

Both pages are rendered directly to a CMYK pixmap (`pymupdf.csCMYK`) at a lower resolution (100dpi by default - `separation_dpi`) and each of the four plates compared independently as its own greyscale image. This is what catches a K-only black rebuilt as a CMYK "rich black", or two colours that render identically in RGB but come from completely different ink mixes - invisible on a monitor, different on a press.

Layers (compare_layers)

PDF Optional Content Groups (OCGs) - the mechanism behind toggleable layers, commonly used for varnish or foil plates - are enumerated via PyMuPDF's OCG API (`doc.get_ocgs()`) and their default-visibility state compared between files. An OCG silently switched off is invisible in a simple render diff (the layer just isn't drawn) but is exactly the kind of change a print operator needs to know about before it reaches a press.

Braille (_braille_dots, _decode_braille, compare_braille)

Braille dots are small filled circles - vector drawing commands, not text. `_braille_dots()` reads the PDF's own path-fill operators looking for small round shapes, clusters them into 2x3 cell groups by position (`_point_cluster()`), then `_decode_braille()` maps each 6-dot pattern to a grade-1 English letter via a fixed lookup table - genuinely decoding the dots to text on both sides, not just diffing dot positions.

12. imaging.py - the JPG/PNG conversion, in full

The entire module, because it's short and every line matters (see the companion document, section 6, for why the naive approach segfaults):

proofer/imaging.py - the whole file, minus docstrings

```
_PNG = b"\x89PNG\r\n\x1a\n"
_JPEG = b"\xff\xd8\xff"

def is_pdf(data: bytes) -> bool:
    return data.lstrip()[:5] == b"%PDF-"

def is_image(data: bytes) -> bool:
    return data.startswith(_PNG) or data.startswith(_JPEG)

def to_pdf_bytes(data: bytes) -> bytes:
    with pymupdf.open(stream=data, filetype="png" if data.startswith(_PNG) else "jpeg") as doc:
        return doc.convert_to_pdf()

def open_pdf(path: str):
    data = Path(path).read_bytes()
    if not is_pdf(data) and is_image(data):
        data = to_pdf_bytes(data)
    return pymupdf.open(stream=data, filetype="pdf")
```

Detection is by magic bytes, not file extension - a renamed file can't fool it either way. `convert_to_pdf()` is PyMuPDF's own method: it places the decoded image as a normal image XObject on a normal one-page PDF, so every downstream consumer - compare(), annotate.build(), the CLI, batch mode - sees a completely ordinary PDF and never needs to know the input wasn't one.

13. proofer/ocr.py - what RapidOCR is actually doing

proofer/ocr.py

```
def _get():
    from rapidocr_onnxruntime import RapidOCR
    return RapidOCR()          # lazy singleton - only loaded if a proof actually needs it

def read(img_rgb, min_conf=0.5):
    scale = 2.0 if max(h, w) < 400 else 1.0     # tiny crops OCR far better upscaled
    img_rgb = cv2.resize(...); img_rgb = cv2.copyMakeBorder(img_rgb, 12,12,12,12, REPLICATE)
    best = _read_one(img_rgb, min_conf)
    if h > 1.6 * w:                              # probably vertical (side-of-carton) text
        for rot in (CLOCKWISE, COUNTERCLOCKWISE):
            cand = _read_one(rotate(img_rgb, rot), min_conf)
            if cand.score > best.score * 1.15: best = cand
    return best.text
```

RapidOCR itself is three chained ONNX models: a text-*detection* model (finds where text is on the page as rotated boxes), an orientation *classifier* (is this box upside down?), and a text-*recognition* model (turns the cropped box into characters). The bundled recognition model - `ch_PP-OCRv4_rec_infer.onnx` - is trained on a Chinese+English character set only.

Empirically confirmed this session

Tested directly this session: a Russian label rendered as an image and read with this exact function came back as `"XpaHNTb B HeAoCTynHOM ... MecTe"` - the model mapped each Cyrillic glyph to whichever character in its Chinese+English alphabet looks most similar, rather than recognising it as Cyrillic and failing loudly. That's a real, confirmed limitation: today's OCR path is only trustworthy for Latin-script (and Chinese) text. A live-text PDF has no such limit, because compare_language and compare_text read the PDF's own Unicode directly - no OCR involved.

14. Spelling and the copy-deck check

proofer/spelling.py

```
LANGS = ("en", "fr", "de", "es", "it", "pt", "nl")
_checkers = [SpellChecker(language=l, distance=1) for l in LANGS]   # pyspellchecker,
edit-distance 1

def checkable(w):
    # skip short words, ALL-CAPS tokens (acronyms/batch codes), and non-Latin scripts entirely
    return len(w) >= 4 and not w.isupper() and LATIN.fullmatch(w)

def is_known(w, extra):
    return w.casefold() in extra or any(c.known([w.casefold()]) for c in _checkers)
```

`extra` is the union of: your own `dictionary.txt`, every word already present in the reference artwork, and every word in the approved text document - so a brand name or INN only has to be "taught" once and never flags again. Seven independent `SpellChecker` instances (one per language) each get a veto: a word only becomes a finding if *none* of the seven dictionaries recognise it.

proofer/copydeck.py - matching an approved paragraph anywhere on the artwork

Loads paragraphs from .docx (parsed as raw XML inside the zip, see section 1), .txt (one paragraph per line) or .pdf (one paragraph per text block, reusing `_text_blocks` from compare.py). Each approved paragraph is then matched against *every* paragraph on the artwork with the same difflib-ratio approach as `_match_blocks` - deliberately not tied to panel order, since a multi-panel leaflet often repeats or reorders approved text across panels.

15. Profiles: Config, but swappable

proofer/profiles.py

```
PRESETS = {
    "strict":  dict(overrides=dict(delta_e=2.0, move_tol_mm=0.15, barcode_shift_mm=0.3, ...)),
    "lenient": dict(overrides=dict(delta_e=5.0, move_tol_mm=0.5, ...), minor_as_info=True),
    "print_run": dict(overrides={}, variable=True),   # lot/batch/expiry changes -> info, not
defect
}

def make_config(settings, **extra):
    kw = dict(PRESETS[settings["profile"]]["overrides"])
    kw.update(settings.get("overrides") or {})   # per-proof manual overrides win
    return dataclasses.replace(Config(), **kw)
```

The "variable data" rule in `apply()` is a single compiled regex built from the label list ("Lot", "Batch No", "Exp", ...), checked against 40 characters of context after each label; a text finding whose changed value sits right after one of those labels gets its severity swapped to `info` and its kind set to `variable_data` - but only when the print-run profile is active.

16. Batch mode and filename-based pairing

proofer/pairing.py classifies each filename by tokenising it (splitting on spaces/underscores/hyphens/dots) and checking the tokens against three word sets:

```
REF  = {"reference", "ref", "approved", "master", "original", "current", "artwork", ...}
SMP  = {"sample", "proof", "new", "revised", "received", "supplier", "draft", ...}
DECK = {"approvedtext", "copydeck", "deck", "text", ...}   # or a .docx/.txt extension
```

Files are grouped by their remaining tokens after removing the role word and any `v3`/`rev2` version marker - so `carton_12_reference.pdf` and `carton_12_sample.pdf` land in the same group (key `"12"`) regardless of folder or order. If no role word is found anywhere, a numbered-version fallback kicks in: the two highest version numbers of the same base name are paired as reference/sample. Anything that still can't be paired is returned as `unpaired` rather than guessed - a wrong guess here would silently proof the wrong two files against each other.

proofer/batch.py then farms every pair out to a `ProcessPoolExecutor` (real OS processes, since this is a one-shot CLI run rather than a long-lived server), and writes one HTML report per pair plus a combined `summary.csv` and `findings.csv`.

17. Report, export and annotation

- **proofer/report.py** turns a Result into the JSON payload the browser UI renders, and can inline every page image as a base64 `data:` URI to produce a fully offline standalone HTML file - open it anywhere, nothing to fetch.
- **proofer/annotate.py** opens the sample through the same `imaging.open_pdf()` used everywhere else, then for each non-info finding draws a coloured rectangle (`page.draw_rect`) and adds a real PDF text annotation (`page.add_text_annot`) with the finding's message as the comment body - so the output opens in Acrobat with clickable, colour-coded comment bubbles, not just a picture of a comment.

Every code excerpt above was copied from the actual files in this session, then trimmed for length where marked ("...") - none of it was reconstructed from memory. Package versions were read with pip list against this exact environment, not requirements.txt's unpinned names.